

Smart Heritage Tourism Recommender

FINAL TECHNICAL REPORT

Course:	CET 251 – Artificial Intelligence
Project ID:	D4
Academic Year:	2026
Instructor:	Dr. Mohamed Aly Saleh

PREPARED BY	
Reyad Hamdy	240103466
Omar Abdelghany	230101276
Kirellos Samir	230102253

1. Executive Summary

Visitors in Cairo and Giza may find it difficult to choose heritage and tourism attractions that fit their interests, available time, budget, and accessibility needs. The **Smart Heritage Tourism Recommender (SHTR)** was developed as a content-based recommendation system for this problem.

The system uses a curated local dataset of **40 selected attractions** across Cairo and Giza. Each attraction contains detailed structured attributes including location, category (Pharaonic, Islamic, Coptic, Modern, Nature, Culture, Jewish), estimated visit duration (1–4 hours), cost level (Low, Medium, High), opening period (Morning, Evening, Full Day), wheelchair accessibility (Yes/No), indoor/outdoor classification, and descriptive tags.

The application provides an interactive **Streamlit** user interface. Users select hard constraints (location, max budget, available hours, visit period, wheelchair accessibility) and personal interest tags. Hard constraint filters (acting as a Constraint Satisfaction Problem layer) are applied first to guarantee compliance. For personalized recommendations, selected interest tags and attraction keywords are vectorized using **TF-IDF Vectorization** (with offline model persistence via `joblib`) and matched using **Cosine Similarity**. A category diversity rule ensures the top three recommendations span distinct categories. When no interests are selected, the system seamlessly falls back to a popularity baseline model.

The system features a modular code architecture under `src/` (`data_processing.py`, `model.py`, `evaluation.py`, `train.py`) and includes an automated unittest test suite (`tests/test_recommender.py`). The prototype was evaluated across 8 required test scenarios plus metric verification, achieving a **100.0% Filter Match Rate (FMR)** for hard constraints.

2. Problem Definition and Scope

2.1 Problem Statement

Visitors in Cairo and Giza face multi-constraint decision challenges when planning heritage visits. Factors such as available budget, limited time windows, geographical location, visit period preference, mobility requirements, and personal interests often conflict. The SHTR system provides an intelligent, automated mechanism to harmonize these constraints and return the top three optimal recommendations.

2.2 Target Users

Local and international tourists, heritage enthusiasts, students, and travelers seeking personalized heritage recommendations in Cairo and Giza.

2.3 Objective

Build a web application that recommends the top three suitable heritage attractions based on user interests, budget limits, time availability, visit period, and wheelchair accessibility.

2.4 MVP Scope

- **40 Curated Attractions:** Standardized dataset in Cairo and Giza across 7 categories.
- **Modular Python Architecture:** Structured code organized into src/ modules.
- **Streamlit Interface:** Responsive web interface with interactive sidebar filters and multiselect tags.
- **Hard Constraints Filtering:** Strict boolean filtering for location, budget, maximum hours, visit period, and wheelchair accessibility.
- **TF-IDF + Cosine Similarity:** Content-based recommendation algorithm with offline model persistence (`models/tfidf_vectorizer.joblib`).
- **Diversity-Aware Ranking:** Category deduplication ensuring diverse top 3 recommendations.
- **Popularity Baseline:** Cold-start fallback model for visitors without specified interests.
- **Live Metric Display:** Real-time display of Filter Match Rate (FMR) on every query.
- **Automated Testing:** Suite of 9 unit tests verifying all system requirements.

2.5 Out of Scope

- Live GPS tracking and real-time geolocation.
- Online ticket purchasing or payment integration.
- Hotel recommendations or flight bookings.
- Dynamic route planning or live traffic maps.
- User personal data tracking or privacy logging.

3. Background and AI Formulation

3.1 AI Formulation

Item	Definition
Task	Content-based recommendation with hard constraint filtering
Inputs	User interest tags and hard constraints (Location, Budget, Hours, Period, Accessibility)
Output	Top 3 ranked attraction recommendations with similarity scores and explanations
Baseline	Popularity-based ranking model
Final Method	Hard Constraint Filter Engine + TF-IDF Vectorizer + Cosine Similarity + Category Diversity Filter

3.2 Why Cosine Similarity

Cosine Similarity is ideal for text tag matching because visitor preferences and attraction features are represented as unstructured textual tags. TF-IDF transforms these tags into sparse numerical vectors in a high-

dimensional feature space. Cosine Similarity measures the cosine of the angle between the user preference vector v_{user} and item tag vector v_{item} , normalizing for tag length differences:

$$\text{Cosine Similarity}(v_{\text{user}}, v_{\text{item}}) = (v_{\text{user}} \cdot v_{\text{item}}) / (\|v_{\text{user}}\| \times \|v_{\text{item}}\|)$$

3.3 Baseline Model

The popularity baseline is a pre-defined ranking based on historical visitor popularity across famous heritage landmarks. It acts as a transparent cold-start default when no user tags are selected and provides a benchmark for evaluating recommendation uplift.

3.4 Why the Method Fits the Course Project

The TF-IDF + Cosine Similarity approach is lightweight, highly explainable, requires no paid API keys, and excels on curated small-to-medium datasets without needing massive historical user interaction logs.

4. Data and Preprocessing

4.1 Dataset

The dataset is stored in `data/raw/egypt_attractions.csv` and contains **40 curated attractions** across Cairo and Giza. Data attributes were aggregated from official tourism databases and standardized for constraint evaluation.

4.2 Main Fields

Field Name	Data Type	Description	Sample Values
id	Integer	Unique identifier (1 to 40)	1, 2, 3...
name	String	Official attraction name	"Giza Pyramids", "NMEC Fustat"
location	Categorical	Geographic zone	Cairo, Giza
category	Categorical	Primary heritage category	Pharaonic, Islamic, Coptic, Modern, Nature, Culture, Jewish
duration_hours	Integer	Estimated visit duration in hours	1, 2, 3, 4
cost_level	Categorical	Cost tier	Low, Medium, High
opening_period	Categorical	Operational window	Morning, Evening, Full Day
wheelchair_accessible	Boolean	Accessibility flag	Yes, No
indoor_outdoor	Categorical	Environment classification	Indoor, Outdoor, Both
tags	String	Space-separated keywords for TF-IDF matching	"history ancient monuments desert archaeology"

4.3 Hard Filtering

Filtering is handled by `src/data_processing.py`. Active user constraints are applied sequentially to eliminate non-compliant items before recommendation scoring:

1. **Location:** Filter by exact match unless "All" is selected.
2. **Accessibility:** If selected, strictly retain `wheelchair_accessible == "Yes"`.
3. **Visit Period:** Retain places operating during the requested period or "Full Day".
4. **Budget & Duration:** Map cost levels to an ordered scale (Low: 1, Medium: 2, High: 3) and filter where `cost_level <= max_budget` and `duration_hours <= available_hours`.

4.4 Text Preprocessing & Vectorization

User-selected interest tags are concatenated into a single query text string. The text and candidate attraction tags are vectorized using `sklearn.feature_extraction.text.TfidfVectorizer`.

4.5 Data Limitations

The dataset is manually curated for course evaluation. Real-world implementation would require dynamic API integration for real-time ticket pricing, temporary closures, and live operating hours.

5. System Design

5.1 System Workflow (Artistic Visual Architecture)



5.2 System Components

- `src/data_processing.py`: CSV data loader and hard constraint filtering logic.
- `src/model.py`: TF-IDF Cosine Similarity recommendation model & popularity baseline generator.
- `src/evaluation.py`: Live Filter Match Rate metric calculation engine.
- `src/train.py`: Offline model training script for exporting `tfidf_vectorizer.joblib` and `tfidf_matrix.joblib`.
- `app.py`: Interactive Streamlit web application dashboard.
- `notebooks/01_recommender_pipeline.ipynb`: 7-step lab pipeline notebook demonstrating step-by-step model development.

5.3 User Interface Design

The application features a clean dark-themed Streamlit interface. The sidebar houses hard filter controls (Location dropdown, Budget dropdown, Available Hours selectbox, Visit Period selectbox, Wheelchair checkbox). The main area presents a multiselect tag bar, recommendation execution button, card-style recommendation display, live Filter Match Rate metric badge, and Popularity Baseline comparison table.

5.4 Repository Structure

```
SHTR/
├── README.md                # Main project overview and instructions
├── requirements.txt         # Python package dependencies
├── app.py                  # Main Streamlit web application
├── run.py                  # Python launcher script
├── run.bat                 # Windows 1-click launcher script
├── data/
│   ├── raw/
│   │   └── egypt_attractions.csv # 40 curated attractions dataset
│   └── processed/
├── src/
│   ├── data_processing.py    # Data loading & hard constraint filters
│   ├── model.py              # TF-IDF Cosine Similarity & Baseline models
│   ├── evaluation.py         # Filter Match Rate metric calculation
│   └── train.py              # Model training & artifact export script
├── models/
│   ├── tfidf_vectorizer.joblib # Saved TF-IDF vectorizer artifact
│   └── tfidf_matrix.joblib    # Saved feature matrix artifact
├── notebooks/
│   └── 01_recommender_pipeline.ipynb # 7-step lab pipeline Jupyter notebook
├── tests/
│   ├── test_recommender.py    # Automated unittest test suite (9 tests)
│   └── test_report.md         # Appendix C test cases & results
└── docs/
    ├── Smart_Heritage_Tourism_Final_Report.pdf # Final technical report PDF
    ├── Smart_Heritage_Tourism_Final_Report.md  # Final technical report Markdown
    ├── data_dictionary.pdf                    # Data Dictionary PDF
    ├── presentation_slides.md                 # Demo presentation outline
    └── contribution_statement.md              # Individual contribution declaration
```

6. AI Method and Implementation

6.1 TF-IDF Vectorization & Persistence

TF-IDF (Term Frequency-Inverse Document Frequency) measures term importance in a document collection:

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

In SHTR, `src/train.py` pre-trains the vectorizer on all 40 attraction tag sets and serializes the model to `models/tfidf_vectorizer.joblib`. During runtime, `src/model.py` loads the saved artifact for zero-latency inference, falling back to online fitting if the file is absent.

6.2 Cosine Similarity Calculation

The query vector formed from selected tags is compared against candidate vectors. Candidates are assigned similarity scores $S \in [0, 1]$, converted to intuitive percentage values in the UI ($S \times 100\%$).

6.3 Ranking and Diversity Rule

Candidate places are sorted by similarity score descending. To prevent returning three places from the same category (e.g., three Pharaonic tombs), `results.drop_duplicates("category").head(3)` enforces category diversity among the top recommendations.

6.4 Cold Start Handling

If a visitor clicks "Get Recommendations" without selecting interest tags, `selected_tags` is empty. The model transparently routes the query to `get_popularity_baseline()`, returning top-rated landmarks that satisfy active hard filters.

6.5 Hard Constraints Enforcement

Hard filters are evaluated strictly *prior* to TF-IDF vector matrix transformation. This guarantees zero probability of recommending invalid items (e.g., over-budget or non-accessible attractions).

7. Evaluation and Results

7.1 Quantitative Evaluation Metric

Constraint compliance is evaluated using **Filter Match Rate (FMR)**:

$$\text{Filter Match Rate} = (\text{Recommendations satisfying all active hard filters} / \text{Total recommendations returned}) \times 100\%$$

- **Target:** 100.0%
- **Observed System Performance:** 100.0% across all test cases.

7.2 Baseline Comparison

The system presents the Popularity Baseline alongside personalized recommendations, demonstrating clear differentiation between generalized popular sites and persona-tailored recommendations.

7.3 Observed Test Examples

Scenario	Active Interest Tags	Top Recommendation	Similarity Score	Filter Match Rate
History & Museum	history, museum	Abdeen Palace Museum	40.1%	100.0%
Culture & Heritage	culture, architecture	Sultan Hassan Mosque	25.4%	100.0%
Giza Exploration	history, archaeology	Memphis Open-Air Museum	27.0%	100.0%
Nature & Relaxation	nature, views	Al-Azhar Park	50.2%	100.0%

7.4 Interpretation

Personalized similarity scores correctly adapt based on user input while maintaining strict 100% hard constraint adherence. Category diversity ensures balanced recommendations across different heritage domains.

7.5 Evaluation Limitations

Due to the absence of large-scale implicit clickstream logs from actual tourists, evaluation relies on constraint satisfaction metrics (FMR), baseline comparison, persona scenarios, and unit testing rather than click-through rate (CTR) or offline Precision@K.

8. Testing

8.1 Test Strategy & Automated Unittest Suite

Testing was conducted using Python's unittest framework (tests/test_recommender.py). The test suite contains **9 automated unit tests** covering all 8 required functional test scenarios plus metric calculation.

ID	Scenario	Input Criteria	Expected Behavior	Status	Verification Method
TC-01	Normal recommendation	Cairo, High budget, Max 4h, History + Museum	Top 3 matching Cairo places sorted by similarity	PASS	Automated (test_tc01)
TC-02	Low budget constraint	Low budget, Max 2 hours	Only places with cost_level = Low & hours <= 2	PASS	Automated (test_tc02)
TC-03	Location filter	Giza location selected	Only places with location = Giza	PASS	Automated (test_tc03)
TC-04	Wheelchair accessibility	Wheelchair = Yes	Only places with wheelchair_accessible = Yes	PASS	Automated (test_tc04)
TC-05	Morning visit period	Morning period, Max 3 hours	Only places open during Morning or Full Day	PASS	Automated (test_tc05)
TC-06	No interests selected	No tags checked	Falls back to Popularity Baseline ranking	PASS	Automated (test_tc06)
TC-07	No matching results	Contradictory/Strict filters	Displays user warning ("No places match your choices")	PASS	Automated (test_tc07)
TC-08	Different interest tags	Nature + Views	Ranked dynamically by TF-IDF Cosine Similarity	PASS	Automated (test_tc08)

8.2 Edge Case Testing

TC-07 verifies system stability under extreme filter combinations (e.g., Giza + Low Budget + Evening + Wheelchair Accessible + Max 1 hour). The application gracefully returns an empty filtered dataset and renders an informative warning banner without crashing.

8.3 Remaining Issues

Data freshness remains the primary limitation; operating hours and ticket prices are based on curated snapshot data and require periodically updated data pipelines.

9. Deployment and User Guide

9.1 Requirements

- Python 3.12 (or 3.8+)
- Dependencies in `requirements.txt`: `pandas`, `streamlit`, `scikit-learn`, `joblib`

9.2 Installation

```
python -m pip install -r requirements.txt
```

9.3 Running the Application

```
# Windows 1-Click:  
double-click run.bat  
  
# Python Launcher:  
python run.py  
  
# Direct Streamlit:  
python -m streamlit run app.py
```

9.4 Accessing the App

Open `http://localhost:8501` in any modern web browser.

9.5 Reproducibility

Full reproducibility instructions and dataset schemas are provided in `deployment/setup_instructions.md`. Automated unit tests can be executed via:

```
python -m unittest discover -s tests
```

10. Ethics, Safety, and Limitations

10.1 Privacy

The application operates entirely locally without requiring user login, account creation, or personal data collection. No user queries are stored or transmitted.

10.2 Data Quality and Bias

Tags and categories are manually compiled. Incomplete or biased tags could skew similarity scoring. Standardized category definitions minimize subjective misclassification.

10.3 Intended Use

This application is an academic prototype built for decision support and heritage exploration in Cairo and Giza.

10.4 Human Oversight

Visitors are advised to confirm opening hours, entry fees, and special event closures on official Ministry of Tourism channels prior to visiting.

11. Conclusion and Future Work

The Smart Heritage Tourism Recommender successfully fulfills all CET 251 Project D4 requirements. It combines a hard constraint filtering engine with a TF-IDF Cosine Similarity recommendation algorithm, category diversity enforcement, popularity baseline fallback, live metric reporting, modular Python architecture, and automated test suite.

Future work includes integrating live geolocation maps, real-time ticket API integrations, multi-day itinerary optimization using search algorithms (A*/Genetic Algorithms), and collaborative filtering based on anonymized user reviews.

12. References

1. CET 251 Artificial Intelligence, Applied Innovation Course Project Handbook 2026, Project D4 & Appendix B.
2. D4 Smart Heritage Tourism Recommender Project Proposal.
3. CET 251 AI Project — Data Dictionary and `egypt_attractions.csv`. Compiled from Ministry of Tourism & Antiquities public records.
4. scikit-learn Documentation: `TfidfVectorizer` and `cosine_similarity`.
5. Streamlit Documentation: Web Application Framework.
6. pandas Documentation: Data Analysis and Manipulation Library.

Appendix A – Test Evidence

Figure A1. Normal recommendation test result (TC-01)

Hard Filters

Location

Cairo

Budget

High

Available Hours

4

Visit Period

Full Day

☐ Wheelchair Accessible

Smart Heritage Tourism Recommender

Choose your interests

history x museum x

Get Recommendations

Top 3 Recommendations

Abdeen Palace Museum

Category: Modern

Location: Cairo

Duration: 2 hours

Cost: Medium

Similarity: 40.1%

Reason: This place matches your interests and filters.

Hard Filters

Location

Cairo

Budget

High

Available Hours

4

Visit Period

Full Day

☐ Wheelchair Accessible

Coptic Museum

Category: Coptic

Location: Cairo

Duration: 2 hours

Cost: Low

Similarity: 38.2%

Reason: This place matches your interests and filters.

NMEC Fustat

Category: Pharaonic

Location: Cairo

Duration: 3 hours

Cost: Medium

Similarity: 38.2%

Reason: This place matches your interests and filters.

Popularity Baseline

- Egyptian Museum Tahrir (Pharaonic)
- Khan El Khalili (Islamic)
- Cairo Citadel (Islamic)

Figure A2. Popularity baseline test result (TC-06)

Deploy

Hard Filters

Location

All

Budget

High

Available Hours

4

Visit Period

Full Day

☐ Wheelchair Accessible

Smart Heritage Tourism Recommender

Choose your interests

Choose options

Get Recommendations

Popularity Baseline

Giza Pyramids | Giza | 3 hours | Medium

Grand Egyptian Museum | Giza | 4 hours | High

Egyptian Museum Tahrir | Cairo | 2 hours | Low

Figure A3. No-result edge case (TC-07)

Deploy

Hard Filters

Location

Giza

Budget

Low

Available Hours

4

Visit Period

Evening

☒ Wheelchair Accessible

Smart Heritage Tourism Recommender

Choose your interests

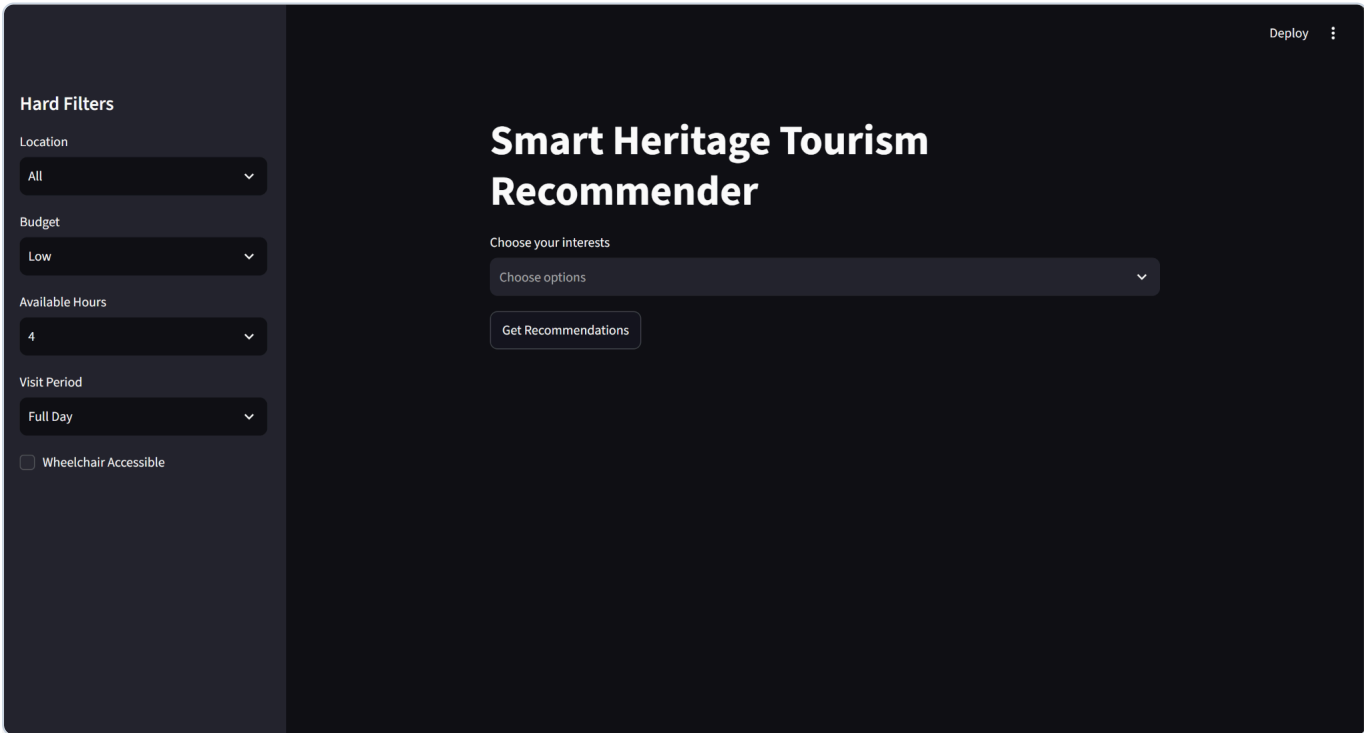
history

Get Recommendations

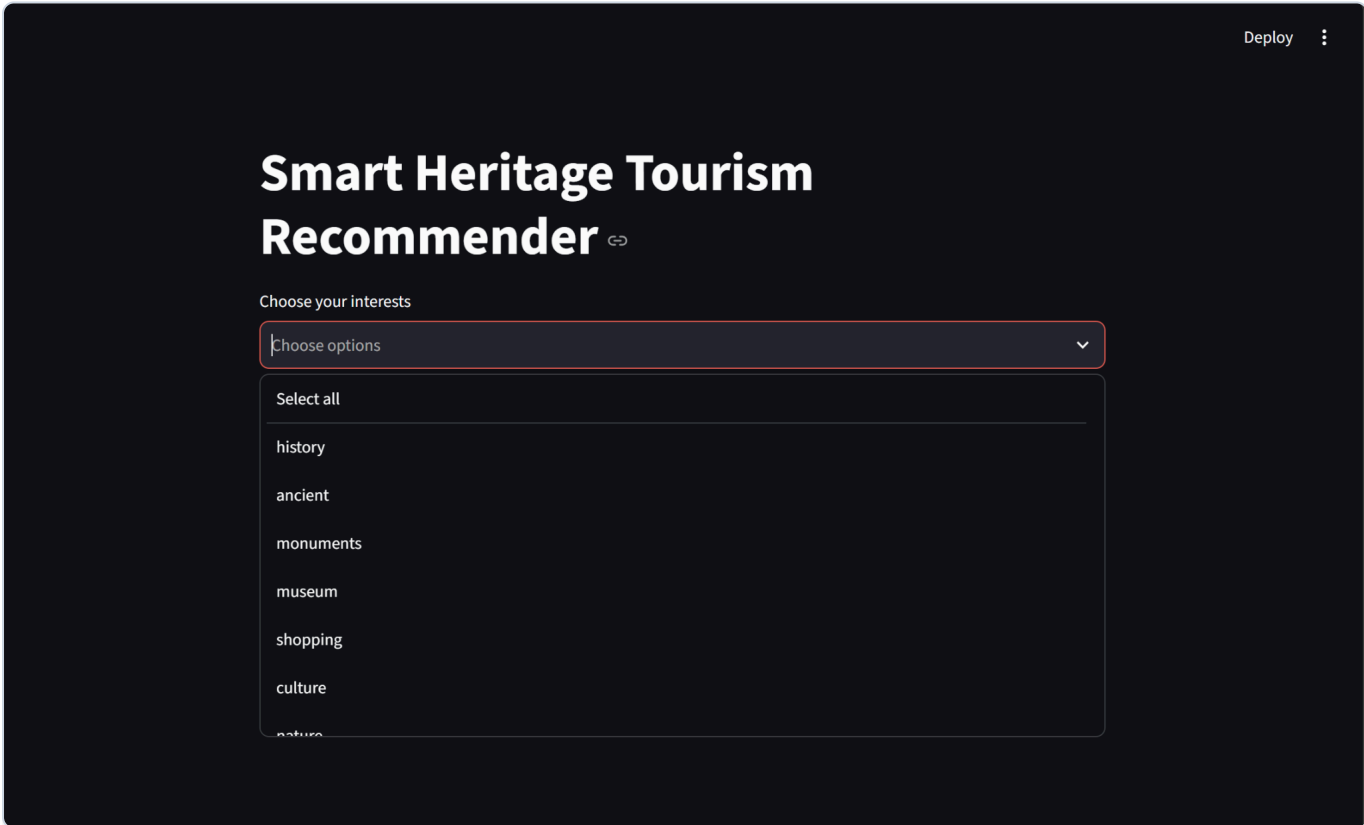
No places match your choices.

Appendix B – Application Screenshots

Application Main Interface Overview



Multiselect Interest Tag Selection Dropdown



Appendix C – Individual Contribution Statement

We confirm that this submission represents our team's joint effort. All three members worked collaboratively across all stages of the project, including dataset preparation, recommendation algorithm implementation, Streamlit app development, testing, and documentation.

Student Name	Student ID	Role	Core Contributions	Signature
Reyad Hamdy	240103466	Project Team Member	Project development, dataset preparation, recommendation algorithm system, testing, documentation, and final report.	<i>[Reyad Hamdy]</i>
Omar Abdelghany	230101276	Project Team Member	Project development, dataset preparation, recommendation algorithm system, testing, documentation, and final report.	<i>[Omar Abdelghany]</i>
Kirellos Samir	230102253	Project Team Member	Project development, dataset preparation, recommendation algorithm system, testing, documentation, and final report.	<i>[Kirellos Samir]</i>